Integration Steps

Steps to integrate the Standard Checkout form on your website.

Follow these steps to integrate the Standard Checkout form on your website:

1. Build Integration

Follow the steps given below:

1.1 Create an Order in Server

Given below are the order states and the corresponding payment states:

Payment Stages	Order State	Payment State	Description
Stage I	created	created	The customer submits the payment information, which is sent to Razorpay . The payment is **not processed** at this stage.
Stage II	attempted	authorized/failed	An order moves from **created** to **attempted** state when payment is first attempted. It remains in this state until a payment associated with the order is captured.
Stage III	paid	captured	After the payment moves to the **captured** state, the order moves to the **paid** state. • No more payment requests are allowed after an order moves to the **paid** state. • The order continues to be in this state even if the payment for this order is **refunded**.

Capture Payments Automatically

You can capture payments automatically with the one-time Payment Capture setting configuration on the Dashboard.

Order is an important step in the payment process.

- An order should be created for every payment.
- You can create an order using the Orders API. It is a server-side API call. Know how to authenticate Orders API.
- The `order_id` received in the response should be passed to the checkout. This ties the order with the payment and secures the request from being tampered.

> ⚠ **Watch Out!**
>
> Payments made without an `order_id` cannot be captured and will be automatically refunded. You must create an order before initiating payments to ensure proper payment processing.

You can create an order using:

API Sample Code | Razorpay Postman Public Workspace

Use this endpoint to create an order using the Orders API.

`POST /orders`

Curl | Java | Python | Go | PHP | Ruby | Node.js | .NET

copy

```
curl -X POST https://api.razorpay.com/v1/orders
-U [YOUR_KEY_ID]:[YOUR_KEY_SECRET]
-H 'content-type:application/json'
-d '{
    "amount": 50000,
    "currency": "USD",
    "receipt": "qwsaq1",
    "partial_payment": true,
    "first_payment_min_amount": 230
}'
```

Success Response | Failure Response

copy

```
{
    "id": "order_IluGWxBm9U8zJ8",
    "entity": "order",
    "amount": 50000,
    "amount_paid": 0,
    "amount_due": 50000,
    "currency": "USD",
    "receipt": "rcptid_11",
    "offer_id": null,
    "status": "created",
    "attempts": 0,
    "notes": [],
    "created_at": 1642662092
}
```

Request Parameters

amount *mandatory*	`integer` The transaction amount, expressed in the currency subunit. For example, for an actual amount of ₹222.25, the value of this field should be `22225` .
currency *mandatory*	`string` The currency in which the transaction should be made. See the list of supported currencies. Length must be of 3 characters.
receipt *optional*	`string` Your receipt id for this order should be passed here. Maximum length is 40 characters.
notes *optional*	`json object` Key-value pair that can be used to store additional information about the entity. Maximum 15 key-value pairs, 256 characters (maximum) each. For example, `"note_key": "Beam me up Scotty"` .
partial_payment *optional*	`boolean` Indicates whether the customer can make a partial payment. Possible values: • `true` : The customer can make partial payments. • `false` (default): The customer cannot make partial payments.
first_payment_min_amount *optional*	`integer` Minimum amount that must be paid by the customer as the first partial payment. For example, if an amount of ₹7,000 is to be received from the customer in two installments of #1 - ₹5,000, #2 - ₹2,000, then you can set this value as `500000` . This parameter should be passed only if `partial_payment` is `true` .

Response Parameters

Descriptions for the response parameters are present in the Orders Entity parameters table.

Error Response Parameters

The error response parameters are available in the API Reference Guide.

1.2 Integrate with Checkout on Client-Side

Add the Pay button on your web page using the checkout code. You can use the handler function or callback URL.

1.2.1 Handler Function or Callback URL

Handler Function	Callback URL
When you use this: • On successful payment, the customer is shown your web page. • On failure, the customer is notified of the failure and asked to retry the payment.	When you use this: • On successful payment, the customer is redirected to the specified URL, for example, a payment success page. • On failure, the customer is asked to retry the payment.

Heads up: callback_url and your webhook URL are different

The callback_url Checkout option is only for WebView and redirect flows. For standard web integrations, use the handler function to handle payment completion on the client side, and webhooks for server-side confirmation. See Webhooks for details.

If you use callback_url in a standard web integration, it bypasses the handler function and the payment status will not be confirmed on the client side immediately after payment.

1.2.2 Code to Add Pay Button

Copy-paste the parameters as `options` in your code:

Callback URL (JS) Checkout Code | Handler Functions (JS) Checkout Code

copy

```
<button id="rzp-button1">Pay</button>
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
<script>
var options = {
    "key": "YOUR_KEY_ID", // Enter the Key ID generated from the Dashboard
    "amount": "50000", // Amount is in currency subunits.
    "currency": "USD",
    "name": "Acme Corp", //your business name
    "description": "Test Transaction",
    "image": "https://example.com/your_logo",
    "order_id": "order_9A33XWu170gUtm", // This is a sample Order ID. Pass the `id` obtained in the response of S
    "callback_url": "https://eneqd3r9zrjok.x.pipedream.net/",
    "prefill": { //We recommend using the prefill parameter to auto-fill customer's contact information especiall
        "name": "John Smith", //your customer's name
        "email": "john.smith@example.com",
        "contact": "+11234567890" //Provide the customer's phone number for better conversion rates
    },
    "notes": {
        "address": "Razorpay Corporate Office"
    },
    "theme": {
        "color": "#3399cc"
    }
};
var rzp1 = new Razorpay(options);
```

```
document.getElementById('rzp-button1').onclick = function(e){
    rzp1.open();
    e.preventDefault();
}
</script>
```

Handy Tips

Test your integration using these test cards.

1.2.3 Checkout Options

key *mandatory*	`string` API Key ID generated from the Dashboard.
amount *mandatory*	`integer` Payment amount in the smallest currency subunit. For example, if the amount to be charged is ₹2,222.50, enter `222250` in this field. In the case of three decimal currencies, such as KWD, BHD and OMR, to accept a payment of 295.991, pass the value as 295990. And in the case of zero decimal currencies such as JPY, to accept a payment of 295, pass the value as 295. **Watch Out!** As per payment guidelines, you should pass the last decimal number as 0 for three decimal currency payments. For example, if you want to charge a customer 99.991 KD for a transaction, you should pass the value for the amount parameter as `99990` and not `99991` .
currency *mandatory*	`string` The currency in which the payment should be made by the customer. See the list of supported currencies.

Handy Tips

Razorpay has added support for zero decimal currencies, such as JPY, and three decimal currencies, such as KWD, BHD, and OMR, allowing businesses to accept international payments in these currencies. Know more about Currency Conversion (May 2024).

name *mandatory*	`string` Your Business/Enterprise name shown on the Checkout form. For example, **Acme Corp**.
description *optional*	`string` Description of the purchase item shown on the Checkout form. It should start with an alphanumeric character.
image *optional*	`string` Link to an image (usually your business logo) shown on the Checkout form. Can also be a **base64** string if you are not loading the image from a network.
order_id *mandatory*	`string` Order ID generated via Orders API.
prefill	`object` You can prefill the following details at Checkout.

Boost Conversions and Minimise Drop-offs

- Autofill customer contact details, especially phone number to ease form completion. Include customer's phone number in the `contact` parameter of the JSON request's `prefill` object. Format: +

(country code)(phone number). Example: "contact": "+919000090000".

- This is not applicable if you do not collect customer contact details on your website before checkout, have Shopify stores or use any of the no-code apps.

notes
optional

`object` Set of key-value pairs that can be used to store additional information about the payment. It can hold a maximum of 15 key-value pairs, each 256 characters long (maximum).

theme

`object` Thematic options to modify the appearance of Checkout.

modal

`object` Options to handle the Checkout modal.

subscription_id
optional

`string` If you are accepting recurring payments using Razorpay Checkout, you should pass the relevant `subscription_id` to the Checkout. Know more about Subscriptions on Checkout.

subscription_card_change
optional

`boolean` Permit or restrict customer from changing the card linked to the subscription. You can also do this from the hosted page. Possible values:

- `true` : Allow the customer to change the card from Checkout.
- `false` (default): Do not allow the customer to change the card from Checkout.

recurring
optional

`boolean` Determines if you are accepting recurring (charge-at-will) payments on Checkout via instruments such as emandate,

	paper NACH and so on. Possible values: • `true` : You are accepting recurring payments. • `false` (default): You are not accepting recurring payments.
callback_url *optional*	`string` Customers will be redirected to this URL on successful payment. Ensure that the domain of the Callback URL is allowlisted.
redirect *optional*	`boolean` Determines whether to post a response to the event handler post payment completion or redirect to Callback URL. `callback_url` must be passed while using this parameter. Possible values: • `true` : Customer is redirected to the specified callback URL in case of payment failure. • `false` (default): Customer is shown the Checkout popup to retry the payment with the suggested next best option.
customer_id *optional*	`string` Unique identifier of customer. Used for: • Local saved cards feature. • Static bank account details on Checkout in case of Bank Transfer payment method.
remember_customer *optional*	`boolean` Determines whether to allow saving of cards. Can also be configured via the Dashboard. Possible values: • `true` : Enables card saving feature. • `false` (default): Disables card saving feature.
timeout *optional*	`integer` Sets a timeout on Checkout, in seconds. After the specified time limit, the customer will not be able to use Checkout.

	⚠ **Watch Out!** Some browsers may pause `JavaScript` timers when the user switches tabs, especially in power saver mode. This can cause the checkout session to stay active beyond the set timeout duration.
readonly	`object` Marks fields as read-only.
hidden	`object` Hides the contact details.
send_sms_hash *optional*	`boolean` Used to auto-read OTP for cards and netbanking pages. Applicable from Android SDK version 1.5.9 and above. Possible values: • `true` : OTP is auto-read. • `false` (default): OTP is not auto-read.
allow_rotation *optional*	`boolean` Used to rotate payment page as per screen orientation. Applicable from Android SDK version 1.6.4 and above. Possible values: • `true` : Payment page can be rotated. • `false` (default): Payment page cannot be rotated.
retry *optional*	`object` Parameters that enable retry of payment on the checkout.
config *optional*	`object` Parameters that enable checkout configuration. Know more about how to configure payment methods on Razorpay standard checkout.

1.2.4 Errors

Given below is a list of errors you may face while integrating with checkout on the client-side.

Error	Cause	Solution
The id provided does not exist.	Occurs when there is a mismatch between the API keys used while creating the `order_id` / `customer_id` and the API key passed in the checkout.	Make sure that the API keys passed in the checkout are the same as the API keys used while creating the `order_id` / `customer_id` .
Blocked by CORS policy.	Occurs when the server-to-server request is hit from the front end instead.	Make sure that the API calls are made from the server side and not the client side.

1.2.5 Configure Payment Methods *(Optional)*

Multiple payment methods are available on the Razorpay Web Standard Checkout.

- The payment methods are **fixed** and cannot be changed.
- You can configure the order or make certain payment methods prominent. Know more about configuring payment methods.

1.3 Handle Payment Success and Failure

The way the Payment Success and Failure scenarios are handled depends on the Checkout Sample Code you used in the last step.

Checkout with Handler Function

If you used the sample code with the handler function:

Payment Success | Payment Failure

On Payment Success

The customer sees your website page. The checkout returns the response object of the successful payment (**razorpay_payment_id**, **razorpay_order_id** and **razorpay_signature**). Collect these and send them to your server.

Checkout with Callback URL

If you used the sample code with the callback URL:

Payment Success | Payment Failure

On Payment Success

Razorpay makes a POST call to the callback URL with the **razorpay_payment_id**, **razorpay_order_id** and **razorpay_signature** in the response object of the successful payment. Only successful authorisations are auto-submitted.

1.4 Store Fields in Your Server

A successful payment returns the following fields to the Checkout form.

Success Callback

- You need to store these fields in your server.
- You can confirm the authenticity of these details by verifying the signature in the next step.

Success Callback

copy

```
{
  "razorpay_payment_id": "pay_29QQoUBi66xm2f",
  "razorpay_order_id": "order_9A33XWu170gUtm",
  "razorpay_signature": "9ef4dffbfd84f1318f6739a3ce19f9d85851857ae648f114332d8401e0949a3d"
}
```

razorpay_payment_id	`string` Unique identifier for the payment returned by Checkout **only** for successful payments.
razorpay_order_id	`string` Unique identifier for the order returned by Checkout.
razorpay_signature	`string` Signature returned by the Checkout. This is used to verify the payment.

1.5 Verify Payment Signature

This is a mandatory step to confirm the authenticity of the details returned to the Checkout form for successful payments.

To verify the `razorpay_signature` returned to you by the Checkout form:

Generate Signature on Your Server

Post Signature Verification

> **Watch Out: Always verify the payment signature server-side**
>
> After a payment completes, Razorpay sends `razorpay_payment_id` , `razorpay_order_id` and `razorpay_signature` to your handler. You must verify the signature on your server before fulfilling the order.
>
> A failed signature check indicates a potentially fraudulent or tampered payment. Reject the order entirely — do not fulfil it, do not retry and do not treat it as a genuine payment. Skipping this step is the most common cause of fraudulent orders and payment disputes.

Here are the links to our

SDKs for the supported platforms.

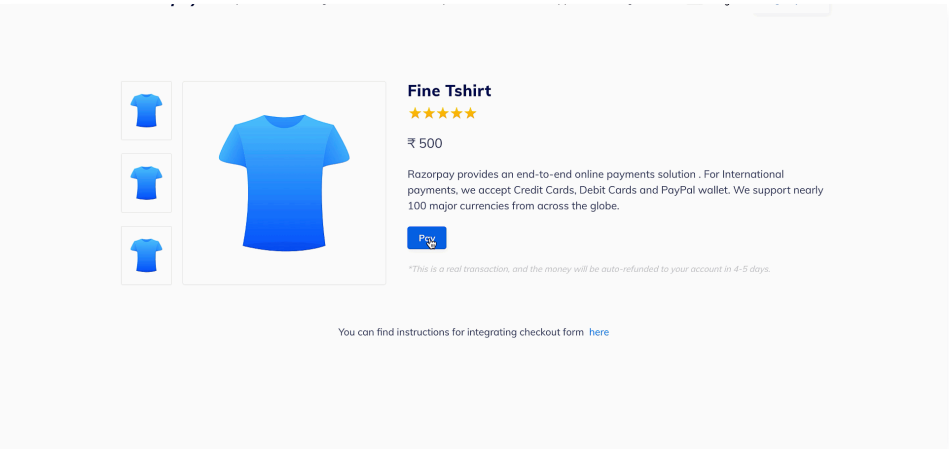
1.6 Verify Payment Status

> **Handy Tips**
>
> On the Razorpay Dashboard, ensure that the payment status is `captured` . Refer to the payment capture settings page to know how to capture payments automatically.

You can track the payment status in three ways:

2. Test Integration

After the integration is complete, a **Pay** button appears on your webpage/app.



Click the button and make a test transaction to ensure the integration is working as expected. You can start accepting actual payments from your customers once the test transaction is successful.

> ⚠ **Watch Out!**
>
> This is a mock payment page that uses your test API keys, test card and payment details.
>
> - Ensure you have entered only your Test Mode API keys in the Checkout code.
> - Test mode features a mock bank page with **Success** and **Failure** buttons to replicate the live payment experience.
> - No real money is deducted due to the usage of test API keys. This is a simulated transaction.

Following are all the payment modes that the customer can use to complete the payment on the Checkout. Some of them are available by default, while others may require approval from us. Raise a request from the Dashboard to enable such payment methods.

Payment Method	Code	Availability
Debit Card	`debit`	✓
Credit Card	`credit`	✓
Netbanking	`netbanking`	✓
UPI	`upi`	✓
EMI - Credit Card EMI , Debit Card EMI and No Cost EMI	`emi`	✓
Wallet	`wallet`	✓
Cardless EMI	`cardless_emi`	Requires Approval .
Bank Transfer	`bank_transfer`	Requires Approval and Integration.

Payment Method	Code	Availability
Emandate	`emandate`	Requires Approval and Integration.
Pay Later	`paylater`	Requires Approval .

You can make test payments using one of the payment methods configured at the Checkout.

Netbanking

You can select any of the listed banks. After choosing a bank, Razorpay will redirect to a mock page where you can make the payment `success` or a `failure` . Since this is Test Mode, we will not redirect you to the bank login portals.

Check the list of

supported banks.

UPI

You can enter one of the following UPI IDs:

- `success@razorpay` : To make the payment successful.
- `failure@razorpay` : To fail the payment.

Check the list of

supported UPI flows.

> **Handy Tips**
>
> You can use **Test Mode** to test UPI payments, and **Live Mode** for UPI Intent and QR payments.

Cards

You can use the following test cards to test transactions for your integration in Test Mode.

Domestic Cards

Use the following test cards for Indian payments:

Network	Card Number	CVV & Expiry Date
Visa	4100 2800 0000 1007	Use a random CVV and any future date
Mastercard	5500 6700 0000 1002	
RuPay	6527 6589 0000 1005	
Diners	3608 280009 1007	
Amex	3402 560004 01007	

Error Scenarios

Use these test cards to simulate payment errors. See the

complete list of error test cards with detailed scenarios. Check the following lists:

- Supported Card Networks.
- Cards Error Codes.

International Cards

Use the following test cards to test international payments. Use any valid expiration date in the future in the MM/YY format and any random CVV to create a successful payment.

Card Network	Card Number	CVV & Expiry Date
Mastercard	5555 5555 5555 4444 5105 1051 0510 5100 5104 0600 0000 0008	Use a random CVV and any future date
Visa	4012 8888 8888 1881	

Check the list of

supported card networks.

Wallet

You can select any of the listed wallets. After choosing a wallet, Razorpay will redirect to a mock page where you can make the payment `success` or a `failure`. Since this is Test Mode, we will not redirect you to the wallet login portals.

Check the list of

supported wallets.

3. Go-live Checklist

Check the go-live checklist for Razorpay Web Standard Checkout integration. Consider these steps before taking the integration live.

3.1 Accept Live Payments

Perform an end-to-end simulation of funds flow in the Test Mode. Once confident that the integration is working as expected, switch to the Live Mode and start accepting payments from customers.

> ⚠ **Watch Out!**
>
> Ensure you are switching your test API keys with API keys generated in Live Mode.

To generate API Keys in Live Mode on your Razorpay Dashboard:

1. Log in to the Razorpay Dashboard and switch to **Live Mode** on the menu.
2. Navigate to **Account & Settings** → **API Keys** → **Generate Key** to generate the API Key for Live Mode.
3. Download the keys and save them securely.
4. Replace the Test API Key with the Live Key in the Checkout code and start accepting actual payments.

Watch Out: Swap your API keys before going live

Your Test Mode API keys only process simulated transactions — no real money moves. If you go live without replacing them with Live Mode keys, customers will see a payment success screen but no payment will be captured or settled.

Generate your Live Mode keys from Dashboard → **Account & Settings** → **API Keys** → **Generate Key** (switch to Live Mode first).

3.2 Payment Capture

After payment is `authorized`, you need to capture it to settle the amount to your bank account as per the settlement schedule. Payments that are not captured are auto-refunded after a fixed time.

Watch Out

- You should deliver the products or services to your customers only after the payment is captured. Razorpay automatically refunds all the uncaptured payments.
- You can track the payment status using our Fetch a Payment API or webhooks.

Auto-capture Payments (Recommended) | Manually Capture Payments

Authorized payments can be automatically captured. You can auto-capture all payments

using global settings on the Razorpay Dashboard. Know more about capture settings for payments.

Watch Out!

Payment capture settings work only if you have integrated with Orders API on your server side. Know more about the Orders API.

Watch Out: Authorised payments must be captured to settle

A payment in `authorized` state has been approved by the bank but not yet settled to your account. You must capture it — either manually from the Dashboard or automatically via capture settings.

Uncaptured payments are auto-refunded after a fixed period. If customers are paying but you are not seeing settlements, check your capture settings in Dashboard → **Account & Settings** → **Payment Capture**.

3.3 Set Up Webhooks

Ensure you have
set up webhooks in the live mode and configured the events for which you want to receive notifications.

Implementation Considerations

Webhooks are the primary and most efficient method for event notifications. They are delivered asynchronously in near real-time. For critical user-facing flows that need instant confirmation (like showing "Payment Successful" immediately), supplement webhooks with API verification.

Recommended approach

- Rely on webhooks for all automation, which can be asynchronous.
- If a critical user-facing flow requires instant status, but the webhook notification has not arrived within the time mandated by your business needs, perform an immediate API Fetch call (Payments, Orders and Refunds) to verify the status.